String class in Java

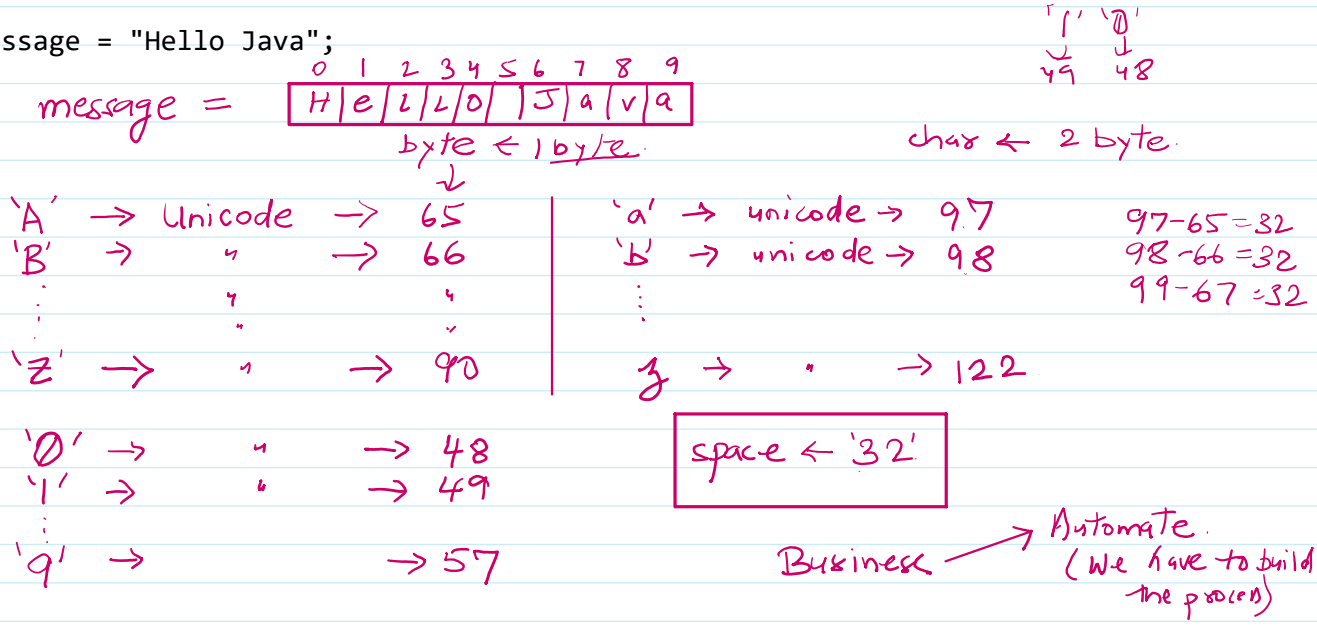
13 December 2025 09:57

What is a String in Java?

In Java, a String is an object that represents a sequence of characters. Unlike primitive data types (such as int, float, etc.) String belongs to the java.lang package and is implemented as a class.

For example:

```
String message = "Hello Java";
```





Key Characteristics:

- **Strings are objects, not primitive types.**
- **Strings are immutable. (cannot be changed once created.)**
- **Java provides rich built-in methods to manipulate strings.**

- Strings are immutable. (cannot be changed once created.)
- Java provides rich built-in methods to manipulate strings.

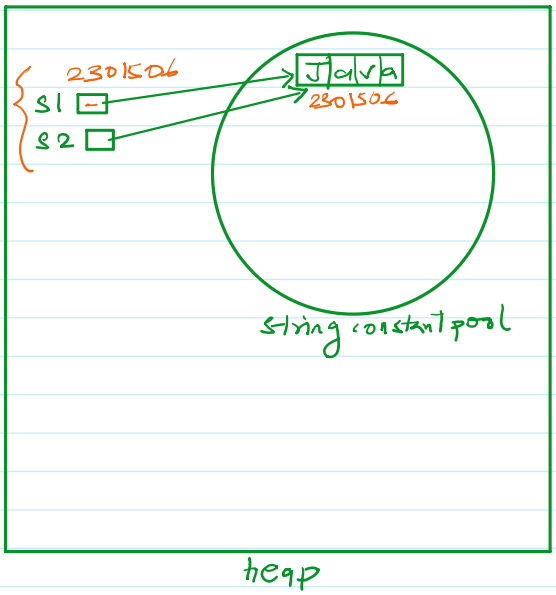
Creating Strings in Java

Using String Literal

```
String s1 = "Java";
```

- Stored in the String Constant pool.
- If the same literal already exist, Java reuses it.

here s1 and s2 is pointing to same memory location.



String Pool: Java maintains a pool of string literals to help save memory. When a new string literal is created, Java checks the Pool for a matching string. If found, the new variable references the pooled string. If not, the new string is added to the Pool.

From <https://www.tpointtech.com/java-string>

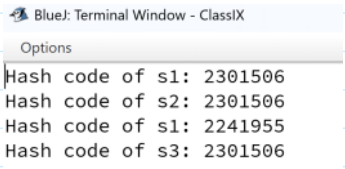
```
package StringClasses;


/**
 * Write a description of class TestStringLiteral here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestStringLiteral
{
    public static void main(String[] args){
        String s1 = "Java";
        String s2 = "Java";
        System.out.print("\fHash code of s1: "+s1.hashCode());
        System.out.print("\nHash code of s2: "+s2.hashCode());
```

it generates the values on the basis of data.

```
        s1 = "Hawa";
        System.out.print("\nHash code of s1: "+s1.hashCode());
        String s3 = "Java";
        System.out.print("\nHash code of s3: "+s3.hashCode());
    }
}
```

Output:



Using new Keyword

```
String s3 = new String("Java");
```

- Creates a new object in heap memory.
- Even if the value is the same, the reference is different.

String immutability

Once a String object is created, its value cannot be modified.

```
package StringClasses;


/**
 * Write a description of class TestStringLiteral here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestStringLiteral
{
    public static void main(String[] args){

        /*String s2 = "Java";
        System.out.print("\fs1"+System.identityHashCode(s1));
        System.out.print("\ns2"+System.identityHashCode(s2));
        String s3 = "Java";
        System.out.print("\n"+System.identityHashCode(s3));

        System.out.print("\n==========================");

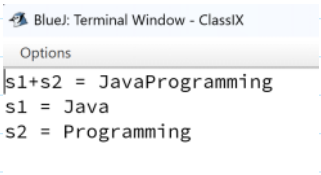
        String s4 = new String("Java");
        String s5 = new String("Java");
        String s6 = new String("Java");
        String s7 = new String("Java");
        System.out.print("\nHash code of s4: "+System.identityHashCode(s3));
        System.out.print("\nHash code of s5: "+System.identityHashCode(s3));
```

```
        System.out.print("\nHash code of s6: "+System.identityHashCode(s3));
        System.out.print("\nHash code of s7: "+System.identityHashCode(s3));*/

        String s1 = "Java";
        String s2 = "Programming";

        System.out.print("\fs1+s2 = "+s1.concat(s2)); // joining of two string
        System.out.print("\ns1 = " + s1 + "\ns2 = " + s2);
    }
}
```

Output:



Reason: concat() creates a new String object, but s1 and s2 refers to the old one.

Advantages of immutability:

- Improved security.
- Safe for multi-threading
- Efficient memory uses via SCP(String Constant Pool).